



Nom i Cognoms:	Erik Rojas Aguilar
URL Repositori Github:	

ACTIVITAT

Objectius:

- Familiaritzar-se amb el desenvolupament d'APIs REST utilitzant Express.js
- Aprendre a integrar serveis de processament de llenguatge natural i visió artificial
- Practicar la implementació de patrons d'accés a dades i gestió de bases de dades
- Desenvolupar habilitats en documentació d'APIs i logging
- Treballar amb formats JSON i processament de dades estructurades

Criteris d'avaluació:

- Cada pregunta indica la puntuació corresponent

Entrega:

- Repositori git que contingui el codi que resol els exercicis i, en el directori "doc", aquesta memòria resposta amb nom "memoria.pdf"

Punt de partida

<https://github.com/jpala4-ieti/DAM-M06-UF04-PR4.2-24-25-Punt-Partida.git>



Preparació de l'activitat

- Clonar el repositori de punt de partida
- Llegir els fitxers README.md que trobaràs en els diferents directoris
- Assegurar-te de tenir una instància de MySQL/MariaDB funcionant
- Tenir accés a una instància d'Ollama
- Completar els quatre exercicis proposats
- Lliurar el codi segons les instruccions d'entrega



Exercicis

Exercici 1 (2.5 punts)

L'objectiu de l'exercici és familiaritzar-te amb **xat-api**. Respon la les preguntes dins el requadre que trobaràs al final de l'exercici.

Configuració i Estructura Bàsica:

1. Per què és important organitzar el codi en una estructura de directoris com controllers/, routes/, models/, etc.? Quins avantatges ofereix aquesta organització?
2. Analitzant el fitxer server.js, quina és la seqüència correcta per inicialitzar una aplicació Express? Per què és important l'ordre dels middlewares?
3. Com gestiona el projecte les variables d'entorn? Quins avantatges ofereix usar dotenv respecte a hardcodejar els valors?

API REST i Express:

1. Observant chatRoutes.js, com s'implementa el routing en Express? Quina és la diferència entre els mètodes HTTP GET i POST i quan s'hauria d'usar cadascun?
2. En el fitxer chatController.js, per què és important separar la lògica del controlador de les rutes? Quins principis de disseny s'apliquen?
3. Com gestiona el projecte els errors HTTP? Analitza el middleware errorHandler.js i explica com centralitza la gestió d'errors.

Documentació amb Swagger:

1. Observant la configuració de Swagger a swagger.js i els comentaris a chatRoutes.js, com s'integra la documentació amb el codi? Quins beneficis aporta aquesta aproximació?
2. Com es documenten els diferents endpoints amb els decoradors de Swagger? Per què és important documentar els paràmetres d'entrada i sortida?
3. Com podem provar els endpoints directament des de la interfície de Swagger? Quins avantatges ofereix això durant el desenvolupament?

Base de Dades i Models:

1. Analitzant els models Conversation.js i Prompt.js, com s'implementen les relacions entre models utilitzant Sequelize? Per què s'utilitza UUID com a clau primària?
2. Com gestiona el projecte les migracions i sincronització de la base de dades? Quins riscos té usar `sync()` en producció?
3. Quins avantatges ofereix usar un ORM com Sequelize respecte a fer consultes SQL directes?

Logging i Monitorització:

1. Observant logger.js, com s'implementa el logging estructurat? Quins nivells de logging existeixen i quan s'hauria d'usar cadascun?
2. Per què és important tenir diferents transports de logging (consola, fitxer)? Com es configuren en el projecte?



3. Com ajuda el logging a debugar problemes en producció? Quina informació crítica s'hauria de loguejar?

Configuració i Estructura Bàsica:

1. Organització del codi:

- És important per mantenir el codi net, modular i fàcil de mantenir. Separar els fitxers en directoris com `controllers/`, `routes/` i `models/` facilita la llegibilitat i el desenvolupament en equips. Això permet canvis localitzats i redueix la complexitat.

2. Seqüència d'inicialització a `server.js`:

- L'ordre correcte per inicialitzar una aplicació Express és:
 1. Configuració de la base de dades
 2. Configuració de middlewares (com `express.json()` o `expressLogger`)
 3. Definició de les rutes
 4. Gestió d'errors amb un middleware global
- L'ordre dels middlewares és important perquè, per exemple, la validació de dades ha de passar abans que la lògica de negoci.

3. Gestió de les variables d'entorn:

- El projecte utilitza el paquet `dotenv` per carregar variables d'entorn des d'un fitxer `.env`. Això evita hardcodejar valors sensibles o configuracions específiques de l'entorn en el codi, i facilita la configuració en diferents entorns (desenvolupament, producció).

API REST i Express:

1. Routing en Express:

- Es fa amb mètodes com `app.get()` o `app.post()`, definint l'endpoint i la funció que es cridarà quan s'accedeixi a aquesta ruta.
- GET s'utilitza per obtenir dades, mentre que POST s'utilitza per enviar dades al servidor per crear o modificar recursos.

2. Separació de la lògica del controlador:

- Separar la lògica del controlador ajuda a seguir el principi de responsabilitat única. Això fa que cada part de l'aplicació sigui més fàcil de mantenir i testeable, aplicant principis de disseny com la modularitat i la reutilització de codi.

3. Gestió d'errors HTTP:

- El middleware `errorHandler.js` centralitza la gestió d'errors. Detecta tipus d'errors específics (com errors de validació de Sequelize o duplicacions), els registra i retorna una resposta adequada al client, amb un codi d'estat HTTP



correcte.

Documentació amb Swagger:

1. Configuració de Swagger:

- A `swagger.js`, es defineixen les opcions per a Swagger, com el títol de l'API, la versió, la descripció i els servidors on s'executa l'API (en aquest cas, en el servidor de desenvolupament local a `http://127.0.0.1:3000`).

2. Documentació amb Swagger:

- Swagger es configura a `swagger.js` per generar la documentació automàticament a partir de comentaris a les rutes (per exemple, `chatRoutes.js`).
- Documentar els paràmetres d'entrada i sortida permet entendre clarament què es necessita i què es retorna en cada endpoint, millorant l'ús i manteniment de l'API.

3. Provar endpoints amb Swagger:

- Swagger permet provar els endpoints directament des de la seva interfície. Això és útil per desenvolupar i depurar l'API, ja que proporciona una manera ràpida de veure les respostes dels endpoints sense necessitat de crear un client extern.

Base de Dades i Models:

1. Relacions entre models a Sequelize:

- La relació entre `Conversation` i `Prompt` es fa amb `belongsTo` i `hasMany`, indicant que una conversa té molts prompts i un prompt pertany a una conversa.
- S'utilitza `UUID` com a clau primària per garantir un identificador únic i difícil de predir, millorant la seguretat.

2. Migracions i sincronització de la base de dades:

- El projecte utilitza Sequelize per gestionar les migracions i la sincronització de la base de dades. L'ús de `sync()` en producció pot ser arriscat, ja que pot provocar pèrdues de dades si es realitzen canvis inesperats a les taules de la base de dades.

3. Avantatges d'usar un ORM com Sequelize:

- Permet treballar amb bases de dades mitjançant objectes en lloc de fer consultes SQL directes. Això simplifica el codi, millora la seguretat (evita SQL Injection) i facilita la migració entre diferents sistemes de bases de dades.



Logging i Monitorització:

1. Logging estructurat amb Winston:

- El logging es fa amb el paquet `winston`, que permet enregistrar missatges amb un format estructurat (incloent-hi informació addicional com el temps i la pila d'errors).
- Els nivells de logging com `info`, `warn` i `error` s'utilitzen per gestionar la gravetat dels missatges, ajudant a classificar la informació.

2. Transports de logging:

- El projecte configura transports de logging per la consola i fitxers, permetent veure logs en temps real i guardar-los en fitxers per a la persistència i anàlisi posterior.
- Això és útil per mantenir registres d'activitat i errors en entorns de producció.

3. Ajuda del logging en la depuració:

- El logging ajuda a depurar problemes en producció, registrant sol·licituds HTTP, errors i altres dades rellevants. Això permet identificar ràpidament els problemes i entendre millor el comportament de l'aplicació.



Exercici 2 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici2.js**

Modifica el codi per tal que, pels dos primers jocs i les 2 primeres reviews de cada jocs crei una estadística que indiqui el nombre de reviews positives, negatives o neutres.

Modifica el prompt si cal.

Guarda la sortida en el directori data amb el nom **exercici2_resposta.json**

Exemple de sortida

```
{
  "timestamp": "2025-01-09T12:30:45.678Z",
  "games": [
    {
      "appid": "730",
      "name": "Counter-Strike 2",
      "statistics": {
        "positive": 1,
        "negative": 0,
        "neutral": 1,
        "error": 0
      }
    },
    {
      "appid": "570",
      "name": "Dota 2",
      "statistics": {
        "positive": 1,
        "negative": 1,
        "neutral": 0,
        "error": 0
      }
    }
  ]
}
```



Exercici 3 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici3.js**

Modifica el codi per tal que retorni un anàlisi detallat sobre l'animal.

Modifica el prompt si cal.

La informació que volem obtenir és:

- Nom de l'animal.
- Classificació taxonòmica (mamífer, au, rèptil, etc.)
- Hàbitat natural
- Dieta
- Característiques físiques (mida, color, trets distintius)
- Estat de conservació

Guarda la sortida en el directori **data** amb el nom **exercici3_resposta.json**

```
{
  "analisi": [
    {
      "imatge": {
        "nom_fitxer": "nom_del_fitxer.jpg",
      },
      "analisi": {
        "nom_comu": "nom comú de l'animal",
        "nom_cientific": "nom científic si és conegut",
        "taxonomia": {
          "classe": "mamífer/au/rèptil/amfibi/peix",
          "ordre": "ordre taxonòmic",
          "familia": "família taxonòmica"
        },
        "habitat": {
          "tipus": ["tipus d'hàbitats"],
          "regioGeografica": ["regions on viu"],
          "clima": ["tipus de climes"]
        },
        "dieta": {
          "tipus": "carnívor/herbívor/omnívor",
          "aliments_principals": ["llista d'aliments"]
        },
        "caracteristiques_fisiques": {
          "mida": {
            "altura_mitjana_cm": "altura mitjana",
            "pes_mitja_kg": "pes mitjà"
          },
          "colors_predominants": ["colors"],
          "trets_distintius": ["característiques"]
        },
        "estat_conservacio": {
          "classificacio_IUCN": "estat",
          "amenaces_principals": ["amenaces"]
        }
      }
    }
  ]
}
```



Exercici 4 (2.5 punts)

Implementa un nou endpoint a xat-api per realitzar anàlisi de sentiment

Haurà de complir els següents requisits

- Estar disponible a l'endpoint POST /api/chat/sentiment-analysis
- Disposar de documentació swagger
- Emmagatzemar informació a la base de dades
- Usar el logger a fitxer

Abans d'implementar la tasca, explica en el requadre com pla plantejaràs i fes una proposta de json d'entrada, de sortida i de com emmagatzemaràs la informació a la base de dades.

Proposta de JSON

Entrada (request body):

```
{ "text": "M'encanta aquest servei! És espectacular." }
```

Sortida (response):

```
{  
  "text": "M'encanta aquest servei! És espectacular.",  
  "sentiment": "positiu",  
  "score": 0.8,  
  "timestamp": "2025-02-19T12:34:56Z"  
}
```

Estructura a Base de Dades (SentimentAnalysis):

Camp	Tipus	Descripció
id	UUID	Clau primària única
text	TEXT	Text analitzat
sentiment	STRING	(positiu, negatiu, neutre)
score	FLOAT	Valor de sentiment (-1 a 1)
createdAt	TIMESTAMP	Data i hora de l'anàlisi